

Scheduling LLM Inference Under Uncalibrated Heterogeneity

Requirements Specification and Research Hypotheses

Project type: Measurement study (systems)

Team: 3 members

Research window: 6 weeks

Status: Scope-frozen specification. Supersedes the prior pitch and advisor brief.

Amendments: incorporates SCOPE-CHANGE-001 (single inference engine) and SCOPE-CHANGE-002 (priority as a passthrough label), both accepted in Week 1.

0. Scope Statement

This document defines a 6-week measurement study. The scheduler it describes is an instrument for producing measurements. Building a production scheduler is out of scope.

Three constraints follow from the 6-week window and are non-negotiable within it:

1. **The simulator is the primary vehicle for breadth.** Physical hardware can span part of the heterogeneity axis, because a single engine exposes per-node capability as configuration (F-9a). The physical pool therefore contributes more than two or three points on R. It cannot reach node counts beyond five, controlled staleness, or values of R beyond what configuration and the available machines can jointly synthesize. Hardware is used to measure the quantities the simulator would otherwise have to assume.
2. **The measurement harness is the Week-1 deliverable.** Trace replay, deterministic request generation, and the results pipeline are treated as first-class engineering and budgeted accordingly, rather than as scaffolding.
3. **A minimum publishable result is defined and independently achievable (\$7).** If the full study does not land, the project still produces a defensible finding.

1. Research Question

In a pool of consumer machines whose per-node throughput is heterogeneous, non-stationary, and known only through stale estimates, does explicit hardware calibration improve scheduling beyond what live queue depth already reveals, and over what range of heterogeneity does that advantage hold?

Two qualifiers carry the novelty claim and must be defended rather than assumed:

Uncalibrated and drifting. Existing heterogeneous-serving work (split prefill/decode across device classes, edge-cloud collaborative inference, multi-generation datacenter fleets) generally assumes device performance profiles that are known and stable, often decided offline. In the regime studied here the performance model must be estimated online and decays.

Consumer-grade variance. Throughput ratios of 10–100× between pool members, against the roughly 2–5× typical of datacenter hardware generations.

The claim is narrower than “heterogeneous scheduling is unstudied.” It is that the uncalibrated, non-stationary, high-variance corner is under-characterized. The claim requires a literature check against 2024–2025 work before the report is written (§8, R-1).

2. Hypotheses

Each hypothesis is falsifiable and has a defined negative result that is reportable.

H1 — Substitution Hypothesis

Statement. Live queue depth is a partial proxy for hardware capability, since slower nodes accumulate queue. The marginal benefit of explicit tok/s calibration, given queue-awareness, is therefore substantially smaller than its benefit over a queue-blind baseline, and shrinks further as offered load rises.

Formally. In the 2×2 design of §4.1, the interaction term between hardware-awareness and queue-awareness is negative: $(WJSQ - JSQ) < (StaticWeighted - RoundRobin)$.

Significance. If the hypothesis holds, the honest headline is that calibration is largely redundant under load, which contradicts the intuition motivating hardware-aware schedulers. If it fails, calibration carries independent signal and the mechanism needs identifying. Either direction is a reportable finding.

H2 — Non-Monotonic Advantage Hypothesis

Statement. The advantage of hardware-aware routing over queue-aware routing is non-monotonic in heterogeneity ratio R . It rises, peaks, and then declines toward zero as R grows.

Mechanism. As $R \rightarrow \infty$, the optimal policy converges to admission thresholding: never dispatch to nodes below a capability cutoff. A policy that excludes weak nodes behaves like round-robin over the strong subset, which a one-line static rule achieves without any calibration machinery. Hardware-aware routing therefore has a sweet spot in R and is matched by a trivial static rule outside it.

Predicted observable. A peak in $(WJSQ - JSQ)$ at intermediate R , converging toward the $Threshold(T)$ baseline at high R .

Significance. This is the prediction that separates the study from a confirmation that more variance yields more benefit. Absent a result of this shape, the study reports a monotonic trend, which is a weaker outcome.

H3 — Staleness Hypothesis

Statement. Routing quality degrades as the age of a node’s throughput estimate approaches the autocorrelation time τ of that node’s actual throughput under load. The location of the H2 peak shifts toward lower R as staleness increases.

Rationale. Calibration error is multiplicative in node speed, so the cost of misestimation grows with R alongside the benefit of estimation. Above some R , expected loss exceeds expected gain.

Significance. This treats the non-stationarity of llama.cpp throughput as the independent variable rather than as a threat to the method. It also gives an empirical basis for heartbeat frequency, which is otherwise an arbitrary choice.

3. Explicit Non-Goals

The following are out of scope and must not be built. Each was considered and rejected for the stated reason.

Excluded	Reason
Model sharding / pipeline parallelism	Separate research problem. Every node holds a full replica.
Autoscaling	Near content-free with 3–5 owned machines; scaling reduces to draining a node. Node join/leave handling is retained only as needed for churn measurement.
Neural-network or RL scheduling policy	The reward signal (p99) is delayed and noisy, sample requirements exceed the window, and the resulting policy is uninterpretable in a writeup. A parametric cost model captures the same signal analytically.
Cross-pool batching as a distinct mechanism	The engine owns batching internally, and under SCOPE-CHANGE-001 there is one engine across the pool rather than two. Reframed as batch-aware routing (§4.2), a property of dispatch.
Streaming in the base system	Full-response return simplifies retry-on-failure. Streaming is retained only as one measured condition in the churn experiment (§4.5).
Decentralized / peer-to-peer scheduling	Introduces herding under stale gossip; a different research question. Noted as future work.
Semantic output-length prediction	Deferred to §9 (stretch). Requires a third model and its own training-data campaign.
Speculative decoding, energy accounting	Out of window.

4. System Requirements

The system exists to produce measurements. Requirements are stated at the minimum fidelity that supports §2.

4.1 Scheduling Policies — 2×2 Factorial (Core Requirement)

The policy set is a factorial design rather than a ladder. A ladder confounds hardware-knowledge with queue-knowledge and cannot decompose the gain.

	Queue-blind	Queue-aware
Hardware-blind	RoundRobin	JSQ (join-shortest-queue)
Hardware-aware	StaticWeighted (weights from calibration, ignores live queue)	WJSQ (capability-weighted queue depth)

One degenerate baseline is added for H2:

- **Threshold(T)** — round-robin over nodes with calibrated throughput above cutoff T; ignores queue and fine-grained capability.

F-1. All five policies **MUST** be selectable from a single configuration value, with no code change between runs.

F-2. Policies **MUST** be implemented against a common dispatch interface so that policy is the only varying factor in a comparison.

F-3. The scheduler **MUST** log, per request, the policy’s decision inputs (observed queue depths, capability estimates, estimate ages) to permit post-hoc attribution of routing errors.

4.2 Batch-Aware Routing

F-4. Dispatch **MUST** account for the fact that requests co-located on one worker are batched by that worker’s engine. Routing decisions therefore consider the composition of a target’s in-flight set as well as its depth.

F-5. The system **MUST NOT** attempt to construct batches externally or modify engine batching internals.

4.3 Cost Model

F-6. Each node **MUST** have a cost model mapping (prompt_tokens, predicted_output_tokens, live_state) → expected service time.

F-7. The cost model **MUST** be a calibrated lookup table over prompt-length buckets, or a low-parameter regression (≤ 6 parameters). It **MUST** be interpretable and inspectable.

F-8. The system **MUST** support artificially aged estimates: a configurable staleness parameter that serves the scheduler a deliberately outdated cost model. H3 requires this, so it is built as a first-class feature rather than a debug flag.

4.4 Worker and Transport

F-9. Workers **MUST** wrap a single inference runtime, llama.cpp with GGUF quantization, on every node, GPU and CPU alike. No engine internals are reimplemented. Engine and quantization format are held constant so that the heterogeneity ratio R is a property of hardware and node configuration alone. A mixed-engine pool would confound R with an engine effect of unknown magnitude that the hypotheses in §2 cannot decompose.

F-9a. Per-node capability MUST be adjustable through runtime parameters of that single engine: GPU offload fraction (`-ngl`), thread count, and slot count (`--parallel`). R is therefore tunable on physical hardware and not solely in simulation. Throttling MUST be applied to distinct physical machines. Multiple logical nodes MUST NOT be co-located on one host, since co-located nodes contend for PCIe, memory bandwidth, and cache, which would reintroduce as contention the confound this requirement removes.

F-9b. The engine effect MUST be measured once and reported as a magnitude. vLLM (AWQ, same model class) is run on the strongest node at one operating point against an identical replayed trace, and the observed throughput ratio to llama.cpp on that same node is reported as a stated bound on external validity. vLLM is a measured condition rather than a pool member: it participates in no policy comparison and appears in no figure other than the engine-gap result.

F-10. Workers MUST heartbeat live state (queue depth, in-flight count, observed recent tok/s, KV-cache occupancy where exposed) at a configurable interval.

F-11. Responses travel worker — client directly. The scheduler is control-plane only and MUST NOT sit in the response data path.

F-12. A centralized scheduler is an accepted single point of failure. The scheduler performs milliseconds of work per request against seconds of worker service time, so at 3–5 nodes it is not the bottleneck. This tradeoff MUST be stated explicitly in the report.

4.5 Admissible Request Space (Cliff Handling)

Extreme heterogeneity produces a categorical failure rather than a latency tail: a long-context request routed to a CPU node may OOM or exceed any reasonable timeout. Naive tail statistics are degenerate under those conditions.

F-13. The primary study MUST operate over an admissible set: a (prompt, output) length range that every pool node can serve within a stated timeout ceiling. The restricted range MUST be reported alongside results.

F-14. Node admissibility MUST be a hard constraint in every policy, baselines included, so that no policy is penalized for producing infinite-latency outcomes.

F-15. The cliff MUST be characterized separately as a standalone observation. Its location and consequences are reported as a limitation of the naive framing rather than folded into the main tail statistics.

4.6 Measurement Harness (Week-1 Deliverable)

Sweeps require infrastructure, and that infrastructure is budgeted as engineering work.

F-16. Deterministic request generator: seeded arrival process (Poisson with configurable burst structure), configurable prompt/output length distributions, configurable priority mix. The priority mix is a generator capability whose output is a label rather than an input to any scheduling decision (§5.4).

F-17. Trace replay: an identical request trace **MUST** be replayable across policies and across the hardware/simulator boundary.

F-18. Per-stage latency attribution per request: `transport_in`, `dispatch`, `queue_wait`, `service` (`prefill` + `decode`), `transport_out`.

F-19. Results pipeline emitting a tidy per-request record set. All figures are generated by script from these records with no manual steps.

F-20. Every experiment **MUST** be reproducible from a single config file plus a seed.

4.7 Simulator

F-21. A discrete-event simulator **MUST** reproduce the scheduler's dispatch logic using the same policy implementations as the live system (shared code, not a reimplementation).

F-22. The simulator's service-time model **MUST** be parameterized from measured hardware data (§5.1), including a stochastic component fitted to observed throughput variance.

F-23. The simulator **MUST** be validated against live-hardware runs on identical replayed traces. Validation criterion: agreement in p50 and p95 end-to-end latency within a stated tolerance across at least 3 operating points. The tolerance and the observed error **MUST** be reported.

F-24. Sweeps beyond the reach of hardware ($R >$ physical pool ratio, node counts > 5 , controlled staleness) run in the validated simulator, and figures **MUST** be labelled as simulated.

5. Method

5.1 Role of Hardware

Physical runs produce three quantities the simulator would otherwise have to invent:

1. **Cost model parameters** — throughput against prompt length and concurrency, per node class.
2. **Non-stationarity characterization** — the autocorrelation time τ of per-node throughput under sustained load, plus its variance envelope. This parameterizes H3 and stands as a result on its own.
3. **Validation anchors** — real runs at 3 or more operating points for F-23.
4. **The synthesizable range of R** — the span of heterogeneity ratios reachable on real machines through per-node offload, thread, and slot configuration (F-9a). This range is determined in Week 2 and reported; the simulator covers R beyond it.

5.2 Role of the Simulator

Node count, offered load, estimate staleness, and heterogeneity ratios beyond the synthesizable range of §5.1 are swept in simulation. A 3–5 node physical pool cannot span these axes.

5.3 Independent Variables

Variable	Range	Vehicle
Heterogeneity ratio R	$\sim 1\times$ to $100\times$	Simulator (hardware:

Variable	Range	Vehicle
(max/min node throughput)		multiple points via F-9a)
Offered load λ	Below saturation $\rightarrow$ saturated	Both
Estimate staleness s	$0 \rightarrow \text{several} \times \tau$	Simulator
Policy	5 policies (§4.1)	Both
Node count N	$3 - 12$	Simulator (hardware: 3–5)

5.4 Dependent Variables

End-to-end latency p50/p95/p99; queue wait; per-node utilization; routing-error rate (dispatches where an alternative node would have completed materially sooner).

priority is generated (F-16) and carried through the trace and every log record, but no policy in §4.1 reads it, and no priority-conditioned metric is reported. It is a passthrough label, retained so that the trace format remains stable for future work (§9). A priority-conditioned metric would be uninformative here: with priority drawn independently of request length and no policy acting on it, high-priority latency is equal to overall latency by construction.

5.5 Controlled Load Band

Policy differences vanish under saturation: once queue wait dominates, every policy converges. Identifying the load band where dispatch policy measurably changes p99 is a prerequisite step, executed before the main sweeps, and is itself a reportable characterization.

6. Timeline — 6 Weeks

Week	Work	Exit criterion
1	Worker wrapper (llama.cpp) with heartbeat; thin client; single request end-to-end; measurement harness (F-16 – F-20)	One query routed and measured; harness replays a seeded trace and emits a per-request record set
2	Calibration campaign: throughput against prompt length and concurrency, per node. Sweep <code>- ngl</code> , thread count, and <code>- -parallel</code> per machine to establish the synthesizable R range. Non-stationarity measurement (τ , variance envelope). F-9b engine-gap measurement at one operating point	Cost model fitted per node class; τ reported; synthesizable R range reported; engine gap measured; MPR-1 achieved (§7)
3	Multi-node pool; all five policies behind one config; admissible-set	All policies runnable; load band identified; validation anchor data collected

Week	Work	Exit criterion
	determination; live runs at 3+ operating points	
4	Discrete-event simulator sharing policy code; parameterize from Week-2 data; validate against Week-3 runs (F-23)	Simulator agrees with hardware within stated tolerance at 3 points
5	Sweeps: $R \times \text{load} \times \text{staleness} \times \text{policy}$. 2×2 decomposition for H1; R-sweep for H2; staleness-sweep for H3	All figures generated; hypotheses tested
6	Analysis, threats to validity, literature positioning, writeup	Final report

Feature freeze: end of Week 3. No new system capability after this point.

Slack: none. The window assumes no week lost to driver, CUDA, or network configuration failures. If a week is lost, §7 defines what survives.

7. Minimum Publishable Result

Defined so the project produces research even under partial failure.

MPR-1 (achievable by Week 2, hardware only). A characterization of throughput non-stationarity in consumer-grade LLM serving nodes under sustained load: the autocorrelation time τ , the variance envelope, and the implication that any single calibrated tok/s figure is a moving average over a non-stationary process. This stands alone as a measurement contribution, requires no scheduler comparison, and motivates the rest of the study.

MPR-2 (achievable by Week 3–4). The 2×2 decomposition (H1) across the synthesized heterogeneity range of F-9a, reported as a range rather than a single figure, plus the load-band characterization. Answers whether calibration is redundant given queue depth, on real hardware, at limited R.

MPR-3 (full study, Weeks 5–6). H2 and H3: the non-monotonic advantage curve and its shift under staleness, in a validated simulator.

Escalation is strictly ordered. MPR-1 does not depend on MPR-2 or MPR-3.

8. Threats to Validity

These are stated in the report rather than left for the examiner to discover.

ID	Threat	Response
R1	The novelty claim may be overstated; heterogeneous-	Literature check required before writeup. If prior work

ID	Threat	Response
	serving work moved rapidly in 2024–2025	covers the uncalibrated regime, narrow the claim to the specific corner that survives.
R2	Simulator validity: results at high R are simulated, and the simulator is validated only at low R	Report validation error explicitly. Label all simulated figures. Do not claim hardware-grounded results beyond the validated range. The hardware-validated range of R is wider than this specification originally assumed, since capability throttling (F-9a) synthesizes intermediate node classes on real machines.
R3	Non-stationary throughput means any cost model is wrong	Converted into the independent variable via H3 rather than assumed away.
R4	Admissible-set restriction reduces the very variance the study targets	Report the restricted range explicitly; characterize the excluded cliff separately (F-15) rather than concealing it.
R5	Small pool (3–5 nodes, few node classes) limits generality of the fitted cost model	Stated as a limitation. The simulator sweeps node count but inherits the measured node classes.
R6	Policy differences may be statistically indistinguishable outside a narrow load band	Load band identified as a prerequisite step (§5.5); null results reported with the band that produced them.
R7	Centralized scheduler is a single point of failure	Accepted and justified (F-12). A design constraint, and reported as such.
R8	6-week window has no slack	MPR ladder (§7) defines graceful degradation.
R9	A single non-production engine limits external validity: results may not transfer to pools running a paged-KV GPU server	The engine is held constant deliberately, to isolate hardware heterogeneity (F-9). The engine gap is measured once on the strongest node and reported as a magnitude (F-9b), which

ID	Threat	Response
		converts an uncontrolled confound into a stated bound.

9. Deferred (Post-Window)

Retained as future work and explicitly not built: semantic output-length prediction feeding the cost model (a length-bucket classifier whose prediction becomes the `predicted_output_tokens` input to F-6); decentralized dispatch; model sharding; energy accounting; speculative decoding across node classes.

Priority-aware dispatch, and priority correlated with request length. Coupling the priority draw to the length bucket would make head-of-line blocking measurable and policy-dependent; adding a priority-aware policy would make it controllable. Both are out of the 6-week window.

10. Ownership

Member	Primary responsibility
A	Worker wrapper (llama.cpp), capability throttling, heartbeat, cost-model calibration campaign, F-9b engine-gap measurement
B	Scheduler core, five policy implementations, staleness injection, shared policy code for simulator
C	Measurement harness, trace replay, results pipeline, simulator, validation, figures

Ownership marks primary responsibility, not exclusive access. All members can run the full stack.

Specification frozen. Changes to §3 (Non-Goals) or §4 (Requirements) after Week 1 require explicit re-scoping against §6. SCOPE-CHANGE-001 and SCOPE-CHANGE-002 were raised and accepted inside Week 1 and are incorporated above.